

Sensors, Measurements and the Estimation Problem

From Physical Quantities to the Kalman Filter

Roberto Marino, PhD

Computational Modeling for Robotics and Human Computer Interaction

University of Messina - FCRLab

roberto.marino@unime.it

May 12, 2026



UniMe
1548

What do we measure in robots?

The fundamental problem

A robot must **act** in the physical world, but has no direct access to the quantities it needs for control. Measurements are always **indirect, partial and noisy**.

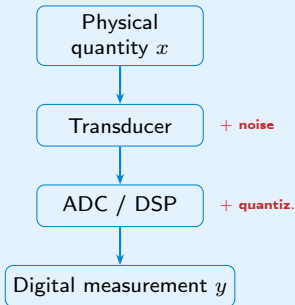
Physical quantities of interest:

- **Kinematic:** position, velocity, acceleration, angles
- **Dynamic:** forces, torques, pressure
- **Electrical:** motor current, voltage
- **Environmental:** distance, temperature, magnetic field

Why is direct state measurement impossible?

- The state includes quantities that are not physically accessible (internal velocity)
- The sensor measures a physical effect *correlated* with the quantity
- The transduction chain introduces distortions and noise

The measurement chain



Taxonomy of robotic sensors

Proprioceptive (internal state)

Measure the internal state of the robot.

Sensor	Measurement
Rotary encoder	joint angle
Linear encoder	linear position
Accelerometer	acceleration
Gyroscope	angular velocity
Current sensors	motor torque

Exteroceptive (external environment)

Measure the world surrounding the robot.

Sensor	Measurement
Lidar	distanza (ToF)
Camera	2D/3D image
Ultrasound	proximity
GPS / GNSS	global position
Force/touch	contact, force

Further classifications

Active / Passive

Active: emits energy (lidar, ultrasound)

Passive: camera, microphone

Contact / Non-contact

Contact: encoder, force sensor

Non-contact: lidar, GPS, camera

Absolute / Incremental

Absolute: knows position from zero

Incremental: measures changes

The mathematical sensor model

General sensor model

$$\mathbf{y}_k = h(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$$

$h : \mathbb{R}^n \rightarrow \mathbb{R}^p$ is the **observation function** (physical sensor model).

$\mathbf{v}_k$ is the **measurement noise**, modelled as zero-mean Gaussian.

Key properties of a real sensor:

- **Range:** measurement interval $[y_{\min}, y_{\max}]$
- **Resolution:** smallest detectable change Δy
- **Accuracy:** how close the mean is to the true value
- **Precision:** how repeatable measurements are (σ)
- **Bandwidth:** maximum measurable frequency [Hz]
- **Latency:** delay between physical event and digital data

The mathematical sensor model

Linear case

If h is linear: $\mathbf{y}_k = \mathbf{C}\mathbf{x}_k + \mathbf{v}_k$

Examples:

- Encoder: $y = \theta + v$ ($\mathbf{C} = [1, 0]$)
- GPS: $y = [x, y]^\top + \mathbf{v}$ ($\mathbf{C} = \mathbf{I}_2$)
- Accelerometer: $y = \ddot{p} + v$

Nonlinear case

Lidar: $y = \sqrt{(x - x_L)^2 + (y - y_L)^2}$

Camera: perspective projection $\neq$ linear

$\Rightarrow$ EKF required (Jacobian of h)

Classification of measurement errors

Systematic errors (deterministic)

Reproducible and predictable. Correctable by calibration.

- **Offset / bias:** $y = x + b$; sensor always reads b too high
- **Gain error:** $y = (1 + \epsilon)x$; incorrect scale
- **Nonlinearity:** response not proportional to input
- **Thermal drift:** bias changes with temperature
- **Quantisation error:** $|e| \leq \Delta/2$ with $\Delta = \text{LSB}$

Random errors (stochastic)

Non-reproducible. Describable only statistically.

- **Thermal noise:** thermal agitation of electrons
- **Shot noise:** current fluctuations
- **Random walk:** random drift of offset over time

Accuracy vs Precision

Accuracy: how close $\mathbb{E}[y]$ is to x

Precision: how small $\text{Var}[y]$ is

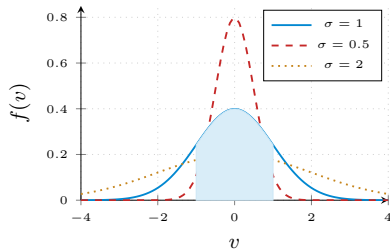
A sensor can be precise but not accurate (bias), or accurate but not precise (high noise).

Probabilistic model of noise

Gaussian: the standard model

$$f(v) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{v^2}{2\sigma^2}\right)$$

- Mean $\mu = 0$ (no systematic bias)
- Variance σ^2 : noise power
- 68% of samples within $[-\sigma, +\sigma]$
- 95% within $[-2\sigma, +2\sigma]$
- 99.7% within $[-3\sigma, +3\sigma]$



Shaded area = 68% of samples (1σ rule).

Central Limit Theorem

The sum of many independent random variables (of any distribution) converges to a Gaussian. Real noise is therefore often well approximated by $\mathcal{N}(0, \sigma^2)$.

IMU: MEMS accelerometer and gyroscope

MEMS accelerometer

Principle: proof mass on springs. Acceleration displaces the mass; displacement is measured capacitively.

Measurement model:

$$\mathbf{a}_m = \mathbf{R}_{WB}^\top (\ddot{\mathbf{p}} - \mathbf{g}) + \mathbf{b}_a + \mathbf{v}_a$$

$\mathbf{b}_a$: bias, $\mathbf{v}_a \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_a)$: white noise. The bias itself is a **random walk**: $\dot{\mathbf{b}}_a = \mathbf{w}_{ba}$.

MEMS gyroscope

Principle: Coriolis effect on a vibrating mass. Measures angular velocity $\boldsymbol{\omega}$.

Measurement model:

$$\boldsymbol{\omega}_m = \boldsymbol{\omega} + \mathbf{b}_g + \mathbf{v}_g$$

- **Angle random walk** N_g [$^\circ/\sqrt{\text{h}}$]: white noise
- **Bias instability** B_g [$^\circ/\text{h}$]: flicker
- Integrating $\boldsymbol{\omega}$ gives the angle: drift **grows over time**

IMU: Acc + gyro fusion

Acc + gyro fusion

Gyroscope: precise in the short term (low noise σ), but drifts long-term (bias random walk).

Accelerometer: no long-term drift, but noisy in the short term and sensitive to vibrations. Fusion via complementary filter or **EKF** $\Rightarrow$ optimal orientation estimate.

Range sensors: lidar and ultrasound

Lidar (Time of Flight)

Principle: emits a laser pulse, measures the return time.

$$d = \frac{c \cdot \Delta t}{2}$$

Measurement models (robot 2D):

$$y_k^{(i)} = \sqrt{(x - x_{W_i})^2 + (y - y_{W_i})^2} + v_k$$

W_i : i -th map point.

- Typical angular resolution: 0.1° – 1°
- Ranging noise: $\sigma \approx 1$ – 5 cm
- Nonlinear in $(x, y) \Rightarrow$ EKF or UKF

Ultrasound

Principle: ToF with acoustic waves (340 m/s). Inexpensive, $\sigma \approx 1$ – 3 cm, but **wide cone** ($\approx 15^\circ$): the detected obstacle is not point-like.

GPS / GNSS

Measurement of global position (x, y, z) by satellite triangulation.

- Accuracy: 2–5 m (standard), < 1 cm (RTK)
- Does not work indoors or under foliage
- $\mathbf{R}_{\text{GPS}} = \text{diag}(\sigma_x^2, \sigma_y^2)$
- No long-term drift $\Rightarrow$ complementary to odometry

Why fuse multiple sensors?

Limits of a single sensor

- **Encoder**: high short-term precision, but accumulated **drift**
- **GPS**: no drift, but imprecise (2–5 m) and does not work indoors
- **Gyroscope**: low noise, but **bias** that when integrated gives growing angular error
- **Accelerometer**: no drift, but very noisy in vibrations

Sensors are **complementary**: the weakness of one is the strength of another.

Advantages of fusion

- **Precision**: fusion reduces variance (more information)
- **Robustness**: if one sensor fails, others compensate

Robotics examples

- Drone: GPS + barometer + IMU → 6DOF pose
- Autonomous car: lidar + camera + radar → perception
- Mobile robot: odometry + lidar → SLAM
- Arm: encoder + F/T sensor → force control

Practical sensor fusion examples in robotics

Odometry + GPS (mobile robot)

- **Odometria:** $\mathbf{Q}$ piccola (basso rumore istantaneo), ma drift $\Rightarrow \mathbf{P}$ cresce nel tempo
- **GPS:** large $\mathbf{R}$ (2–5 m), but no drift
- **Fusion:** GPS periodically corrects odometry drift; between GPS fixes the prediction uses the odometric model

Lidar + Camera (obstacle detection)

Lidar: precise 3D position, no semantic information.
Camera: rich semantics (object class), imprecise position.
Fusion: data association + EKF for object tracking.

GPS + Barometer (UAV)

GPS: 3D position with vertical error ≈ 10 m.
Barometer: relative altitude with $\sigma \approx 0.5$ m.
KF fusion: estimated altitude much more precise than GPS alone.

In practice: ROS 2

Package `robot_localization`: implements an EKF/UKF to fuse arbitrary sensors. Configuration via YAML file: sensor topics and matrices $\mathbf{Q}$, $\mathbf{R}$ are specified.
De facto standard for ROS 2 mobile robots.

Estimation: formal definition and criteria

The estimation problem

Given a probabilistic model $p(\mathbf{y}|\mathbf{x})$ (*likelihood*) and a measurement $\mathbf{y}$, find the **best estimate** $\hat{\mathbf{x}}$ of the unknown quantity $\mathbf{x}$.

Optimality criteria:

ML – Maximum Likelihood:

$$\hat{\mathbf{x}}_{\text{ML}} = \arg \max_{\mathbf{x}} p(\mathbf{y}|\mathbf{x})$$

Maximises the probability of observing $\mathbf{y}$ given $\mathbf{x}$. Does not use prior knowledge of $\mathbf{x}$.

MAP – Maximum a Posteriori:

$$\hat{\mathbf{x}}_{\text{MAP}} = \arg \max_{\mathbf{x}} p(\mathbf{x}|\mathbf{y}) = \arg \max_{\mathbf{x}} p(\mathbf{y}|\mathbf{x})p(\mathbf{x})$$

Incorporates a prior distribution $p(\mathbf{x})$.

MMSE – Minimum Mean Square Error:

Fundamental connections

- With $\mathbf{v} \sim \mathcal{N}(0, \mathbf{R})$ and uniform prior:
ML $\equiv$ Least Squares (LSQ)
- With $\mathbf{v} \sim \mathcal{N}(0, \mathbf{R})$ and Gaussian prior $\mathbf{x} \sim \mathcal{N}(\bar{\mathbf{x}}, \mathbf{P}_0)$:
MAP $\equiv$ weighted LSQ with regularisation
- For linear systems with Gaussians:
MMSE $\equiv$ MAP $\equiv$ KF update step

Recursive estimation — the Kalman precursor

Problem

Measurements arrive sequentially $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_k$. We want to update $\hat{\mathbf{x}}$ **online** without recomputing everything from scratch.

Recursive LSQ (RLS) — $\mathbf{x}$ static, linear model

$$y_k = \mathbf{h}_k^\top \mathbf{x} + v_k:$$

$$\mathbf{P}_k = \mathbf{P}_{k-1} - \frac{\mathbf{P}_{k-1} \mathbf{h}_k \mathbf{h}_k^\top \mathbf{P}_{k-1}}{1 + \mathbf{h}_k^\top \mathbf{P}_{k-1} \mathbf{h}_k}$$

$$\mathbf{K}_k = \mathbf{P}_{k-1} \mathbf{h}_k (1 + \mathbf{h}_k^\top \mathbf{P}_{k-1} \mathbf{h}_k)^{-1}$$

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_{k-1} + \mathbf{K}_k (y_k - \mathbf{h}_k^\top \hat{\mathbf{x}}_{k-1})$$

Identical structure to the Kalman filter!

The estimate updates as:

old estimate + *gain* $\times$ *innovation*.

Fundamental properties of RLS

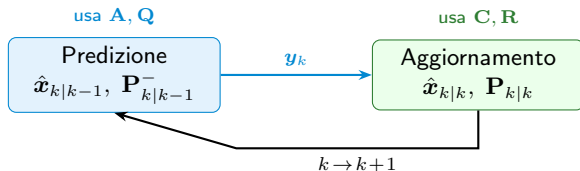
- $\mathbf{P}_k$ **decreases**: each new measurement reduces uncertainty
- $\mathbf{K}_k \rightarrow \mathbf{0}$: with many measurements the gain vanishes (the estimator “trusts” its own estimate)
- Equivalent to batch LSQ with the same measurements

From static to dynamic estimation

Limitation of static estimation

RLS estimates a constant x . But in robotics the state evolves: $x_k = Ax_{k-1} + Bu_k + w_k$. With pure RLS the estimate quickly becomes stale.

Key idea: split the problem into two steps.



- **Prediction:** propagates state with the dynamic model $\Rightarrow P$ increases
- **Update:** corrects with the measurement $\Rightarrow P$ decreases

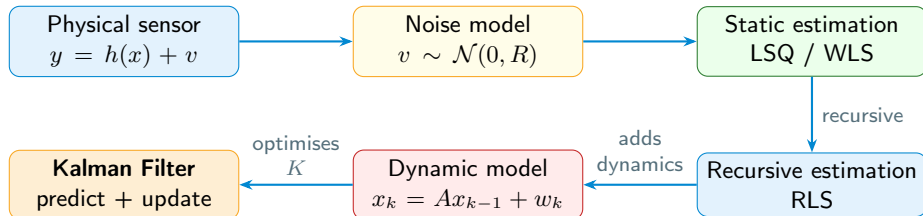
Bayesian interpretation

Prediction = prior $p(x_k | y_{1:k-1})$:
propagates the density through the model.

Update = posterior $p(x_k | y_{1:k})$:
incorporates the likelihood of the new measurement.

The KF is the **exact** solution to this Bayesian recursion for linear systems with Gaussian noise.

Summary — from measurement to dynamic estimation



What we have covered so far

- Physical sensors and their model $y = h(x) + v$
- Noise characterisation: σ^2 , $\mathbf{R}$
- Estimation criteria: ML, MAP, MMSE
- LSQ and RLS: estimation for static x
- Observability: when estimation is feasible

What is still missing

- The **state evolves over time** as $\mathbf{x}_k = \mathbf{A}\mathbf{x}_{k-1} + \mathbf{w}_k$
- Prediction with the model **increases** uncertainty
- The measurement **reduces** it
- The optimal gain $\mathbf{K}$ balances the two effects
- $\Rightarrow$ **Kalman Filter**

The Kalman Filter: From Intuition to Equations

*“When you have two imperfect estimates,
combine them giving more weight to the more reliable one.”
(Rudolf E. Kalman)*

A robot that never fully trusts anything

A robot needs to know **where it is**. It has two sources of information:

Wheel odometry

- Counts steps → estimates position
- Precise in the short term
- **Drift accumulates** over time

GPS / laser sensor

- Directly measures position
- No long-term drift
- **Noisy** at every reading

Both are imperfect. Which one should the robot trust?

The key insight

Neither. The robot should **combine both**, giving more weight to whichever is more reliable *at that moment*.

The optimal weight depends on how uncertain each source is. This is exactly what the Kalman filter computes.

The result

The combined estimate is **more accurate than either source alone**

The uncertainty cloud

The robot never says “*I am at 5 m*”.

It always says “*I am at 5 m, give or take 0.5 m*”.

The Kalman filter tracks **two quantities** at every step:

State and covariance

$\hat{\mathbf{x}}_k$ best estimate of the state

$\mathbf{P}_k$ how uncertain that estimate is

$\hat{\mathbf{x}}_k$ is the **centre** of the uncertainty cloud.

$\mathbf{P}_k$ is its **size** $\hat{\mathbf{a}}$ small when confident, large when unsure.

The cloud through time

- **Prediction step:** cloud *grows*
(model is not perfect $\rightarrow$ uncertainty increases)
- **Update step:** cloud *shrinks*
(sensor brings new information)

The Gaussian assumption

The cloud is modelled as a **Gaussian distribution**:

$$p(\mathbf{x}_k \mid \mathbf{y}_{1:k}) = \mathcal{N}(\hat{\mathbf{x}}_k, \mathbf{P}_k)$$

Two numbers are enough to describe everything we know.

Telling the filter how the world works

Before writing the equations, we must give the filter two models.

How the state evolves (dynamics)

$$\mathbf{x}_k = \mathbf{A}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k + \mathbf{w}_k, \quad \mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$$

What the sensor measures (observation)

$$\mathbf{y}_k = \mathbf{C}\mathbf{x}_k + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$$

- A** physics of motion
- Bu_k** effect of control input
- Q** model imperfection (wheels slip, ...)
- C** what the sensor can see
- R** sensor noise (GPS error, ...)

Robot on a rail: $\mathbf{x} = [p, v]^\top$

$$\mathbf{A} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

GPS sees position only; velocity is *inferred* by the filter.

Step 1 Prediction: the cloud grows

No sensor reading yet use the model to **forecast** where the robot will be.

Predicted state (P1)

$$\hat{\mathbf{x}}_k^- = \mathbf{A}\hat{\mathbf{x}}_{k-1} + \mathbf{B}\mathbf{u}_k$$

"I know where I was and how I move, so I can predict where I am now."

The mean propagates **deterministically** through the nominal model.

Predicted covariance (P2)

$$\mathbf{P}_k^- = \mathbf{A}\mathbf{P}_{k-1}\mathbf{A}^\top + \mathbf{Q}$$

"Every step I take adds a little more doubt: the cloud grows."

$\mathbf{A}\mathbf{P}_{k-1}\mathbf{A}^\top$: previous uncertainty warped by motion.

+ $\mathbf{Q}$: new uncertainty from model imperfection.

The Kalman gain: how much to trust the sensor

The sensor reading has arrived. Before updating, compute **how much weight** to give it.

Kalman gain (K)

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{C}^\top (\mathbf{C} \mathbf{P}_k^- \mathbf{C}^\top + \mathbf{R})^{-1}$$

Scalar case ($n = p = 1$, $C = 1$) â the intuition is clearest here:

$$K_k = \frac{P_k^-}{P_k^- + R} \in [0, 1]$$

$$P_k^- \gg R$$

Prediction uncertain,
sensor precise.

$$K \approx 1$$

Trust the sensor.

$$P_k^- \ll R$$

Prediction precise,
sensor noisy.

$$K \approx 0$$

Trust the model.

$$P_k^- = R$$

Equal uncertainty.

$$K = 0.5$$

Arithmetic mean.

Example: odometry uncertainty $\sigma = 0.2$ m, GPS $\sigma = 0.1$ m $\Rightarrow K = 0.04 / (0.04 + 0.01) = 0.8$. The GPS gets

Step 2, Update: the cloud shrinks

Innovation

$$\boldsymbol{\nu}_k = \mathbf{y}_k - \mathbf{C}\hat{\mathbf{x}}_k^-$$

The **surprise**: difference between what the sensor actually measured and what you expected it to measure. If $\boldsymbol{\nu}_k = \mathbf{0}$, the sensor confirms the prediction, no correction needed.

Updated state (U1)

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k \boldsymbol{\nu}_k$$

"Take the prediction and correct it by K times the surprise."

Updated covariance (U2)

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{C}) \mathbf{P}_k^-$$

"I used the sensor, now I know where I am with less uncertainty than before."

$\mathbf{P}_k \leq \mathbf{P}_k^-$ **always**: every measurement, even a noisy one, reduces uncertainty.

The five equations, the complete filter

Prediction (uses $\mathbf{A}$, $\mathbf{B}$, $\mathbf{Q}$)

$$\hat{\mathbf{x}}_k^- = \mathbf{A}\hat{\mathbf{x}}_{k-1} + \mathbf{B}\mathbf{u}_k$$

(P1 : predicted state)

$$\mathbf{P}_k^- = \mathbf{A}\mathbf{P}_{k-1}\mathbf{A}^\top + \mathbf{Q}$$

(P2 : predicted covariance)

Update (uses $\mathbf{C}$, $\mathbf{R}$)

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{C}^\top (\mathbf{C}\mathbf{P}_k^- \mathbf{C}^\top + \mathbf{R})^{-1}$$

(K : Kalman gain)

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k(\mathbf{y}_k - \mathbf{C}\hat{\mathbf{x}}_k^-)$$

(U1 : updated state)

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{C}) \mathbf{P}_k^-$$

(U2 : updated covariance)

Two inputs you must provide:

Q: how much you trust your model

R: how noisy your sensor is

The filter is optimal: $\mathbf{K}_k$ minimises $\text{tr}(\mathbf{P}_k)$, the total estimation error variance.

The two knobs: **Q** and **R**

R: sensor noise covariance

Has a direct physical meaning.

How to get it:

- Hold the sensor still on a known value
- Record N measurements
- Compute $R = \hat{\sigma}^2$

Or read it from the datasheet.

It is a property of the hardware.

Q: process noise covariance

Represents **how much you trust your model**.

- Good motors, flat ground $\rightarrow$ small **Q**
- Rough terrain, wind, unmodelled forces $\rightarrow$ large **Q**

Often tuned by trial and error.

Large **Q**

Filter trusts measurements more.

Reacts quickly to changes.

Estimate can be noisy.

Small **Q**

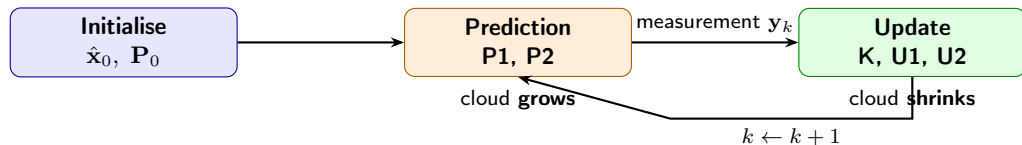
Filter trusts model more.

Estimate is smooth.

Slow to react if reality changes.

The ratio Q/R matters more than their absolute values.

Summary: the Kalman filter in one picture



What the filter needs from you:

- A dynamic model $\mathbf{A}, \mathbf{B}$
- An observation model $\mathbf{C}$
- Process noise $\mathbf{Q}$ (model trust)
- Sensor noise $\mathbf{R}$ (from calibration)
- Initial guess $\hat{\mathbf{x}}_0, \mathbf{P}_0$

What you get:

- Optimal estimate $\hat{\mathbf{x}}_k$ at every step
- Honest uncertainty $\mathbf{P}_k$ at every step
- Velocity estimated even if not directly measured
- Automatic balance between model and sensor

*Used in every autonomous robot, smartphone, drone, and self-driving car.
First used by NASA in the Apollo programme to navigate to the Moon.*